

Numerical Methods for ODEs: Exercise Week 7

Problem 1:

Implement the general explicit two-step method

$$y_{n+2} + a_1 y_{n+1} + a_0 y_n = h (b_1 f_{n+1} + b_0 f_n) \quad (1)$$

for given parameters a_1, a_0, b_1, b_0 and a fixed time step size h for a general initial value problem

$$y'(t) = f(t, y(t)), \quad y(0) = y_0. \quad (2)$$

You can use `odeint` or `solve_ivp` to generate the missing second initial value y_1 . Verify for a suitable test problem that if you use the parameters for the Adams-Bashforth-2 method you see the expected second order convergence.

Solution. A possible implementation can be found in `ex_1.py`

□

Problem 2:

Consider the initial value problem

$$y'(t) = 0, \quad y(0) = 1 \quad (3)$$

and the two-step, third-order method

$$y_{n+2} = -4y_{n+1} + 5y_n + h (2f_n + 4f_{n+1}) \quad (4)$$

derived in the script. As shown there, the method applied to (3) leads to values

$$y_n = A + B (-5)^n. \quad (5)$$

Determine numerically the constants A, B for $y_0 = y(0)$ and $y_1 = 1 + 1e - 15$ and confirm that your implementation from Task 1 gives the same result as (5).

Solution. Consider general initial values y_0 and y_1 . Then, from (5), it follows that

$$y_0 = A + B (-5)^0 = A + B \quad (6)$$

$$y_1 = A + B (-5)^1 = A - 5B. \quad (7)$$

This is a system of two linear equations that can easily be solved. Adding five times the first equation to the second gives

$$6A = 5y_0 + y_1 \Rightarrow A = \frac{1}{6} (5y_0 + y_1). \quad (8)$$

Putting this back into the first equation gives

$$y_0 = \frac{5}{6}y_0 + \frac{1}{6}y_1 + B \Rightarrow B = \frac{1}{6}(y_0 - y_1). \quad (9)$$

A possible implementation can be found in `ex_2.py` □

Problem 3:

Compare the AB-2 multi-step method against the improved Euler method for the van-der-Pol oscillator from Problem 3 in last week's exercise.

Solution. A possible implementation can be found in `ex_3.py`. Both methods perform similarly, with improved Euler giving slightly better results both for small ($\mu = 0.1$) and larger ($\mu = 5.0$) values of μ . This shows that the added work per time step in Runge-Kutta methods is not necessarily an obstacle to good performance: smaller error constants may still make them win out over multi-step methods! □